

Servicios web

SERVICIOS DE RED E INTERNET

Estructura y Funcionamiento de la WWW



Estructura y Funcionamiento de la WWW

- **Hipertexto:** La WWW está basada en el concepto de hipertexto, que permite la interconexión de documentos mediante enlaces. Estos enlaces permiten que los usuarios naveguen entre diferentes páginas web con un simple clic.



Estructura y Funcionamiento de la WWW

- **Protocolo HTTP:** Para facilitar la transferencia de información en la web, se creó el **Protocolo de Transferencia de Hipertexto (HTTP)**, que es el estándar que define cómo los mensajes se formatean y transmiten, y qué acciones deben tomar los navegadores web y los servidores para responder a las diversas solicitudes.



Estructura y Funcionamiento de la WWW

- **HTML (Lenguaje de Marcado de Hipertexto):** Los documentos en la web están escritos en un lenguaje llamado HTML, el cual describe la estructura de la información. Los navegadores web interpretan este lenguaje para mostrar el contenido de una forma legible y atractiva para el usuario (con texto, imágenes, videos, etc.).



Estructura y Funcionamiento de la WWW

- **Navegadores Web:** Los navegadores son herramientas que permiten a los usuarios interactuar con la WWW. Interpretan las páginas escritas en HTML, las solicitudes a través de HTTP y presentan la información de manera visual. Algunos ejemplos de navegadores incluyen Google Chrome, Mozilla Firefox, y Microsoft Edge.



Estructura y Funcionamiento de la WWW

- **Servidores Web:** Los servidores son ordenadores donde se almacenan las páginas web. Cada vez que un usuario solicita una página web mediante su navegador, este envía una solicitud al servidor usando HTTP, y el servidor responde con el contenido de la página solicitada.

Estructura y Funcionamiento de la WWW

- **W3C (World Wide Web Consortium):** A medida que la web fue creciendo, era esencial establecer estándares comunes para asegurar su interoperabilidad. El W3C fue creado en 1994 para desarrollar protocolos y directrices que aseguren que la web continúe siendo abierta, accesible y universal.

MIME

- **MIME (Multipurpose Internet Mail Extensions)** es un estándar que permite a los usuarios de Internet enviar y recibir no solo texto, sino también otros tipos de archivos como imágenes, vídeos, audio, documentos PDF, y más, a través de protocolos como el correo electrónico (SMTP) y la web (HTTP).

MIME

Por qué es importante MIME?

Originalmente, los correos electrónicos solo podían manejar texto simple en formato ASCII (caracteres básicos del inglés), lo que limitaba enormemente la capacidad de compartir archivos o contenido más complejo. MIME surgió para resolver esto, permitiendo:

- **Adjuntar archivos:** Como imágenes, videos, o documentos a un correo electrónico.
- **Transferir texto en varios idiomas:** Soportando caracteres especiales que no están en el conjunto básico de ASCII (como letras acentuadas, símbolos, etc.).
- **Soporte para múltiples tipos de contenido:** Dentro de un mismo mensaje o archivo web.

MIME

Cómo funciona MIME?

- Cuando envías un correo con un archivo adjunto o una página web contiene varios tipos de medios (como imágenes o vídeos), MIME se encarga de describir qué tipo de archivo es, cómo debe procesarse y de asegurarse de que esos archivos puedan transferirse por Internet de manera correcta.
- Lo hace a través de **cabeceras MIME**, que son pequeños fragmentos de información añadidos a los mensajes que le indican al receptor qué tipo de contenido se está enviando.

Cabecera mime

- Content-Type: Esta cabecera dice qué tipo de archivo es (texto, imagen, vídeo, etc.).
- Content-Transfer-Encoding: Especifica cómo está codificado el archivo para que pueda ser transmitido (ya que los datos binarios deben ser convertidos a texto antes de ser enviados por algunos sistemas).

```
HTTP/1.1 200 OK
Date: Wed, 23 Jan 2019 17:37:54 GMT
Expires: -1
Cache-Control: private, max-age=0
Content-Type: text/html; charset=UTF-8
Strict-Transport-Security: max-age=31536000
Server: gws
X-XSS-Protection: 1; mode=block
X-Frame-Options: SAMEORIGIN
Set-Cookie: 1P_JAR=2019-01-23-17; expires=Fri, 22-Feb-2019
17:37:54 GMT; path=/; domain=.google.com
Alt-Svc: quic=":443"; ma=2592000; v="44,43,39"
Connection: close
Content-Length: 246589

<!doctype html><html itemscope=""
itemtype="http://schema.org/WebPage" lang="en"><head><meta
charset="UTF-8"><meta content="origin" name="referrer"><meta
```


Características Principales de MIME

- **Transparencia al Usuario:** MIME actúa en segundo plano, lo que significa que los usuarios no notan su funcionamiento cuando envían o reciben correos electrónicos o al interactuar con páginas web.
- **Uso en Correo Electrónico:** MIME se utiliza ampliamente en el envío de correos electrónicos, permitiendo adjuntar archivos de varios tipos y soportando el uso de caracteres que no están en el conjunto US-ASCII. Los correos electrónicos en Internet suelen llamarse **mensajes SMTP/MIME**, debido a que MIME amplía las capacidades del protocolo SMTP.
- **Extensión a Otros Protocolos:** Aunque originalmente fue diseñado para el correo electrónico, MIME también se utiliza en otros protocolos como HTTP, permitiendo la transferencia de diferentes tipos de contenido en la web.
- **Soporte para Diferentes Idiomas:** Una mejora clave que MIME aporta al correo electrónico es el soporte para diferentes juegos de caracteres (más allá del inglés), lo que facilita la transferencia de textos en múltiples idiomas.

HTML

- HTTP (HyperText Transfer Protocol) es el protocolo que permite la comunicación en la World Wide Web (WWW), permitiendo la transferencia de datos entre un **cliente** (generalmente un navegador web) y un **servidor**. Es la base del funcionamiento de la web tal como la conocemos hoy.



LINEA DE TIEMPO DE HTML

- <https://prezi.com/udxibk5mehm6/linea-del-tiempo-html/>

Características principales de HTTP

- **Protocolo Cliente-Servidor:** En HTTP, el cliente (como tu navegador) envía una **petición** al servidor solicitando un recurso (una página web, una imagen, un video, etc.). El servidor recibe esta petición, la procesa, y responde enviando el recurso solicitado o un mensaje de error si no se encuentra disponible.
- **Protocolo Sin Estado (Stateless):** HTTP es **sin estado**, lo que significa que cada vez que el cliente hace una nueva solicitud al servidor, este no tiene memoria de las interacciones anteriores. No guarda información sobre las conexiones pasadas del cliente. Cada petición es tratada de forma independiente, como si fuera la primera vez que ambos se comunican.
- **No orientado a sesiones:** Debido a que no guarda el estado entre solicitudes, cada vez que el cliente hace una nueva petición, se debe establecer una **nueva conexión** con el servidor. Esto hace que HTTP sea rápido y ligero, pero a veces se requiere mantener sesiones (por ejemplo, cuando inicias sesión en un sitio web). Para solucionar esto, se utilizan tecnologías adicionales como **cookies** o **tokens**.

Características principales de HTTP

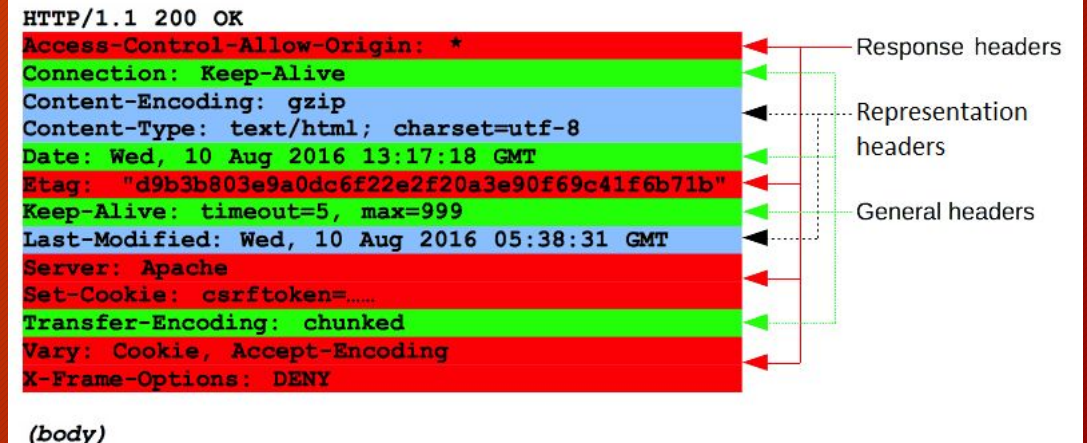
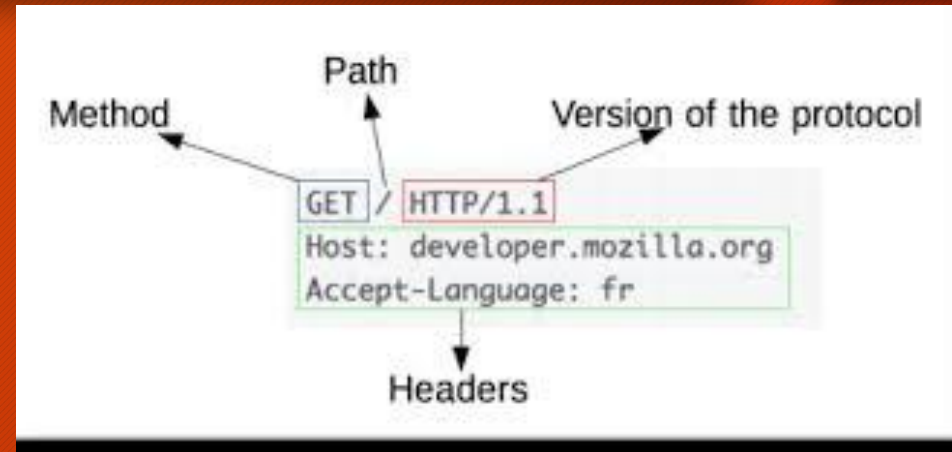
- **Originalmente diseñado para HTML:** HTTP fue diseñado inicialmente para transmitir documentos escritos en **HTML (HyperText Markup Language)**, el lenguaje de marcación que define la estructura y contenido de las páginas web. Sin embargo, hoy en día **HTTP se utiliza para transferir todo tipo de archivos**, como imágenes, vídeos, audios y archivos de aplicación, como JSON o XML.
- **Aplicaciones Modernas:** Aunque el propósito original de HTTP era transmitir páginas web, ahora se utiliza para **aplicaciones web** complejas y otros servicios, como APIs. Además de las páginas web, aplicaciones como las que usas en tu móvil o plataformas en la nube también usan HTTP para comunicarse con servidores y transferir datos.

Estructura de un Mensaje HTTP

Estructura de un Mensaje HTTP:

Línea de Inicio (Start Line):

- **Peticiones (Request):** La línea de inicio de una petición HTTP describe lo que el cliente está solicitando del servidor. Contiene:



Estructura de un Mensaje HTTP

Método HTTP: Indica la acción que el cliente quiere realizar. Algunos ejemplos son:

- **GET:** Solicitar datos (solo lectura).
- **POST:** Enviar datos (para crear o modificar).
- **PUT:** Reemplazar o crear un recurso.
- **DELETE:** Eliminar un recurso.
- **HEAD:** Obtener solo las cabeceras.
- **PATCH:** Modificar parcialmente un recurso.
- **OPTIONS:** Consultar los métodos permitidos.
- **CONNECT:** Establecer un túnel de conexión (por ejemplo, para HTTPS).
- **TRACE:** Realizar una traza de la solicitud (usado para depuración).

 Brij Kishore Pandey

DON'T FORGET TO SAVE

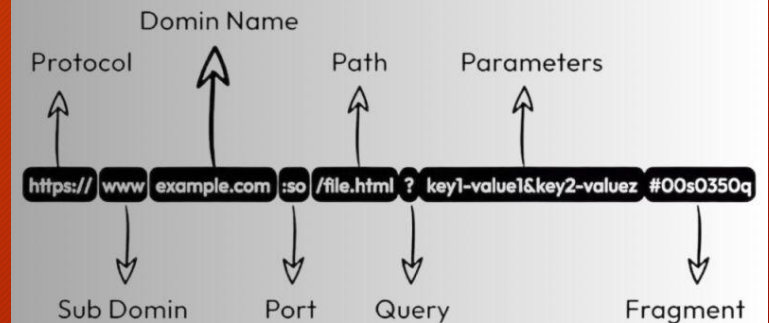
TOP 9 HTTP REQUEST METHODS

GET Example :- <code>GET /api/employees/{employee-id}</code> ↳ Returns a specific employee by Employee ID. Description :- Retrieves data from the server. It should only retrieve data and should have no other effect on the data.	POST Example :- <code>POST /api/employees/department</code> ↳ Creates a department resource Description :- Used to send data to a server to create a new resource. The data is included in the body of the request.	PUT Example :- <code>PUT /api/employees/123</code> ↳ Update employee by employee ID Description :- Similar to POST, but it's used to update an existing resource. The entire resource is updated at once.
PATCH Example :- <code>PATCH /api/employees/123 { "name": "Brij" }</code> ↳ Updates name for employee id 123 Description :- Also used to update resources but it only updates the fields that were included in the request.	DELETE Example :- <code>DELETE /api/employees/235</code> ↳ Delete employee by Employee ID. Description :- The DELETE method deletes a resource. Regardless of the number of calls, it returns the same result.	HEAD Example :- <code>HEAD /api/employees</code> ↳ Similar to GET, but it does not return the list of employees Description :- Typically this is used to check if a resource is available and to get the metadata.
OPTIONS Example :- <code>OPTIONS /api/main.html/1.1</code> ↳ Returns Permitted HTTP Method in this URL Description :- This Method is used to get information about the possible communication options for the given URL or an asterisk to refer to the entire server	TRACE Example :- <code>TRACE /api/main.html</code> ↳ Responds the exact request that client sent. Description :- The TRACE Method is for diagnosis purposes. It echoes the received request so that a client can see what changes or additions have been made by intermediate servers.	CONNECT Example :- <code>CONNECT www.example.com:443 HTTP/1.1</code> ↳ Connects to the URL Provided Description :- The Connect Method is for making end to end connections between a client and a server. It makes a two way connection like a tunnel between them.

URL

Una URL (Uniform Resource Locator) es una dirección única que se utiliza para localizar recursos en la World Wide Web (como páginas web, imágenes, videos, etc.). Cada recurso disponible en la web tiene su propia URL, lo que permite a los navegadores y usuarios acceder directamente a ellos.

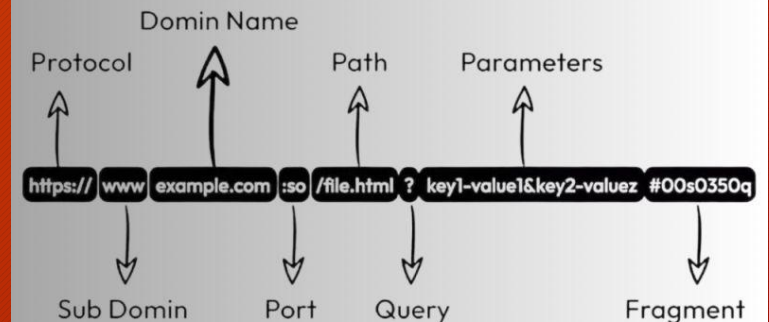
Structure of URL



Componentes de una URL:

- Protocolo (https://):
- El protocolo indica cómo se debe acceder al recurso. En este caso, https (HyperText Transfer Protocol Secure) se usa para realizar la comunicación de forma segura. Otros ejemplos de protocolos son http y ftp.

Structure of URL

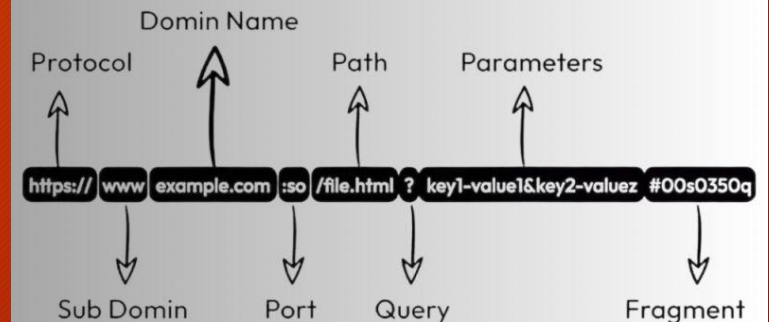


Componentes de una URL:

Nombre de dominio (medac.es):

- El dominio es el nombre del servidor en el que está alojado el recurso. En este caso, medac.es es el dominio, que representa la dirección del servidor web donde se encuentra el sitio de Medac. El dominio está formado por:
 - Nombre: medac
 - Extensión de dominio de nivel superior (TLD): .es, que indica que es un dominio de España.

Structure of URL



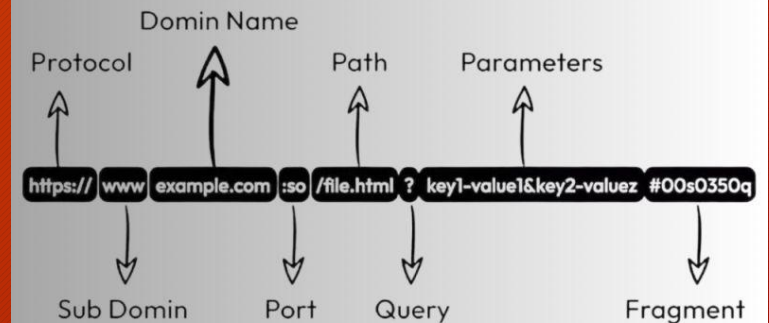
Componentes de una URL:

Ruta del recurso

(/grado-medio/sistemas-microinformaticos-y-redes):

- La ruta indica la ubicación exacta del recurso dentro del servidor. Es como la estructura de carpetas donde se encuentra el archivo o la página en particular.
- En este caso, grado-medio es una carpeta dentro del sitio, y sistemas-microinformaticos-y-redes es el recurso específico (una página, probablemente).

Structure of URL



Otros componentes de una URL

Puerto:

- Especifica el puerto del servidor al que conectarse (por defecto, el puerto 80 para HTTP y 443 para HTTPS). No aparece a menos que sea diferente al predeterminado.
- Ejemplo: <https://medac.es:8080/>

Parámetros de consulta:

- Se añaden después de un signo ? y permiten enviar datos al servidor.
- Ejemplo: <https://medac.es/busqueda?curso=sistemas>

Fragmento o "hash":

- Se añade después de un # y permite referirse a una sección específica dentro de la página.
- Ejemplo: <https://medac.es/grado-medio#seccion-de-informacion>

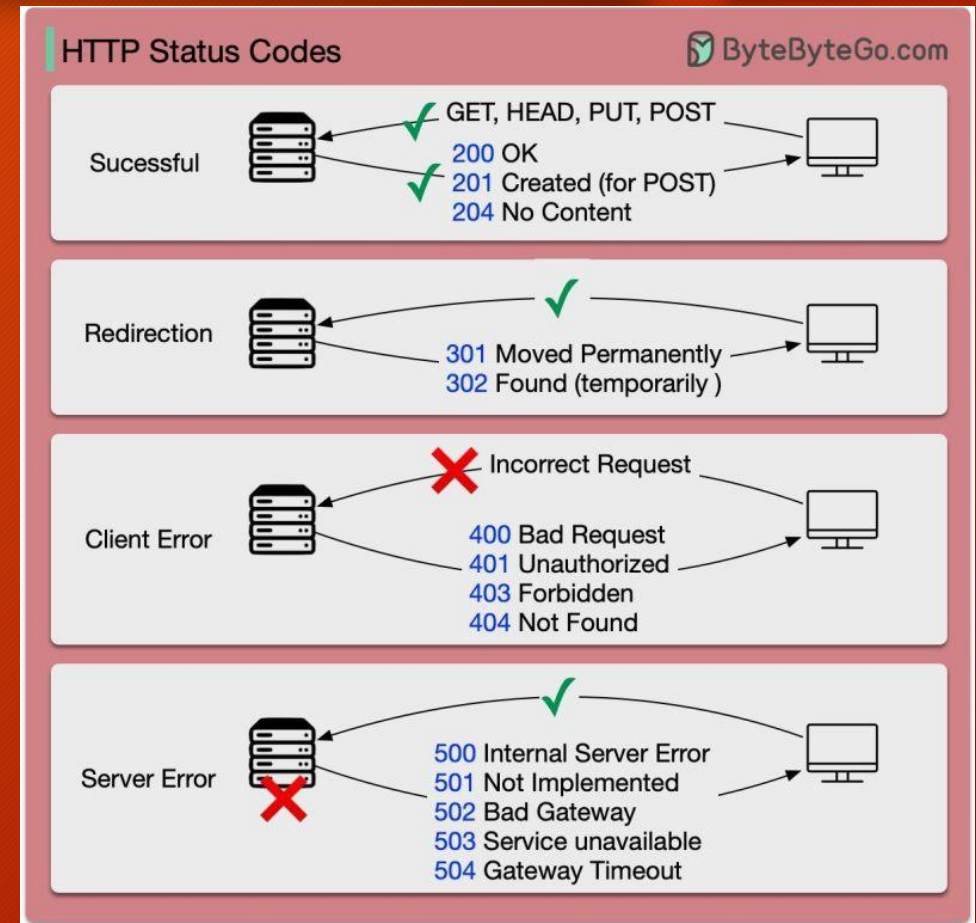
Cabeceras y códigos de estado y error

1xx - Informativos: Indican que la solicitud fue recibida y el proceso sigue en curso.

- 100 Continue: El cliente debe continuar con la solicitud.
- 101 Switching Protocols: El servidor está cambiando de protocolo según lo solicitado por el cliente.

2xx - Éxito: Indican que la solicitud fue recibida, entendida y aceptada correctamente.

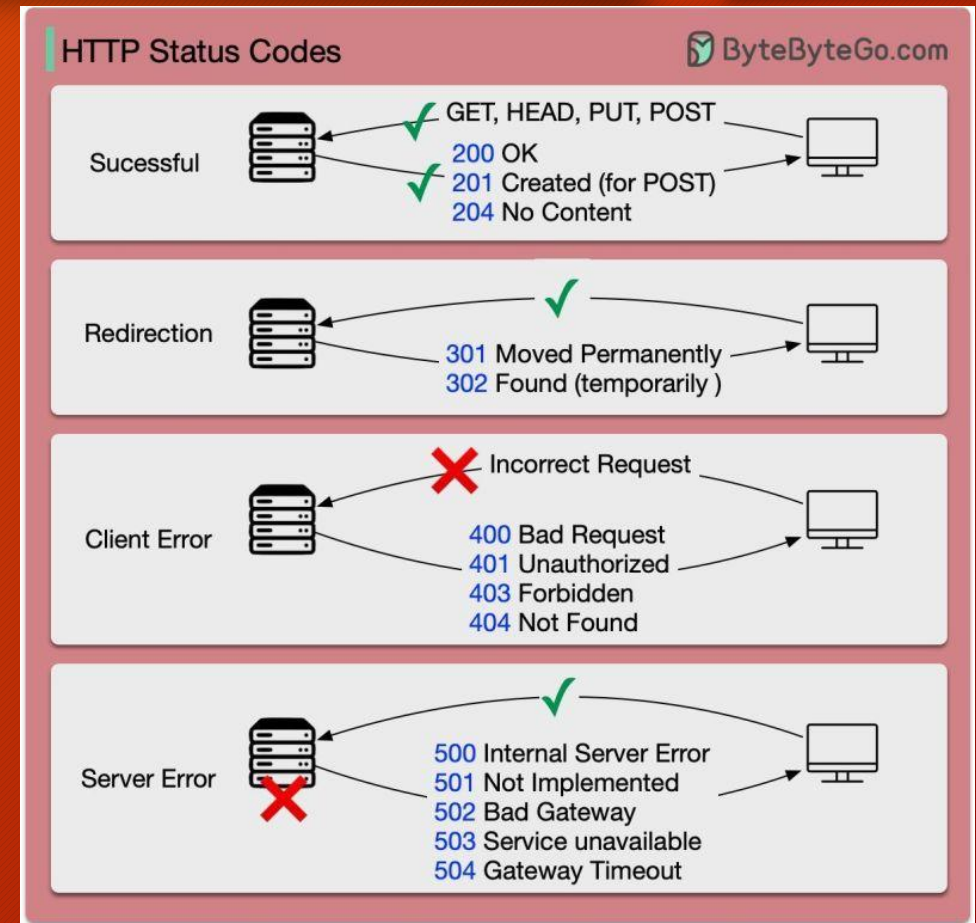
- 200 OK: La solicitud fue exitosa y el recurso se devolvió en el cuerpo de la respuesta.
- 201 Created: El recurso fue creado satisfactoriamente, por ejemplo, después de una solicitud POST.
- 204 No Content: La solicitud fue exitosa, pero no se devuelve ningún contenido.



Cabeceras y códigos de estado y error

3xx - Redirección: Indican que se necesita más acción por parte del cliente para completar la solicitud.

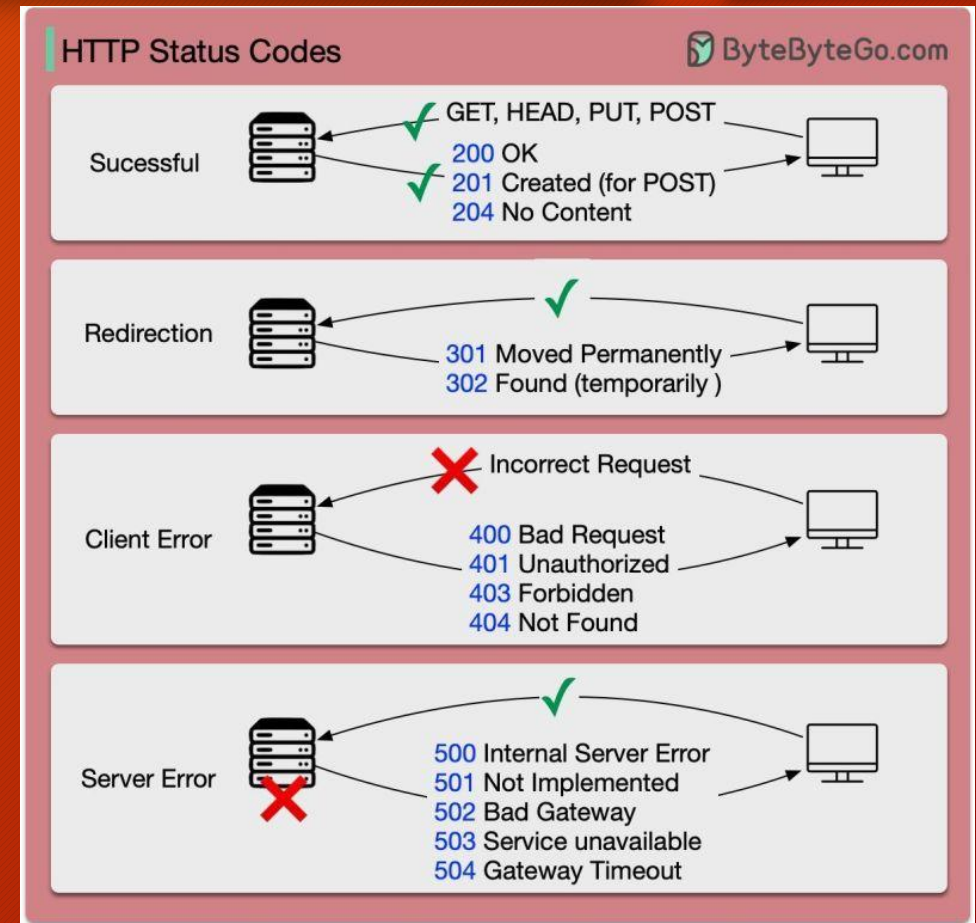
- 301 Moved Permanently: El recurso solicitado se ha movido permanentemente a una nueva URL.
- 302 Found: El recurso se ha movido temporalmente a una nueva URL.
- 304 Not Modified: El recurso no ha cambiado desde la última vez que se solicitó. Se puede usar la versión almacenada en caché.



Cabeceras y códigos de estado y error

4xx - Errores del Cliente: Indican que hubo un error en la solicitud del cliente.

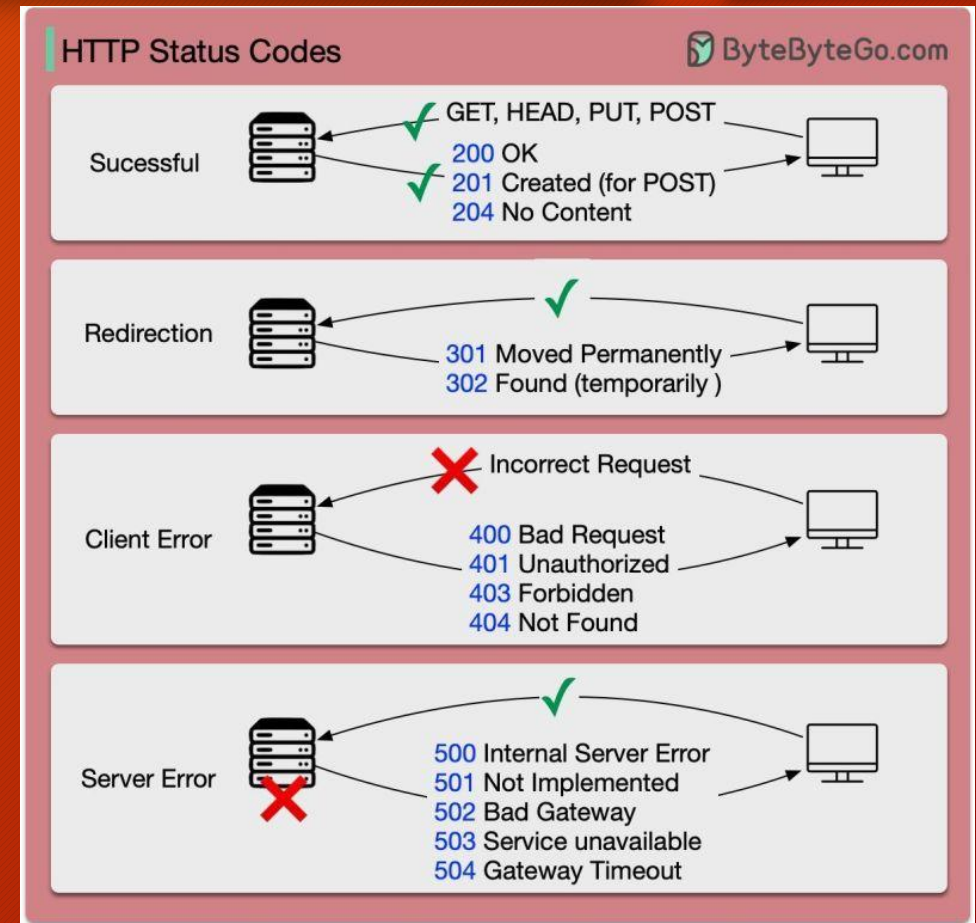
- 400 Bad Request: La solicitud está malformada o no se pudo entender.
- 401 Unauthorized: El cliente debe autenticarse para obtener el recurso.
- 403 Forbidden: El servidor entiende la solicitud, pero se niega a autorizarla.
- 404 Not Found: El recurso solicitado no pudo ser encontrado en el servidor.
- 405 Method Not Allowed: El método HTTP utilizado no está permitido para el recurso solicitado.



Cabeceras y códigos de estado y error

5xx - Errores del Servidor: Indican que el servidor falló al completar una solicitud válida.

- **500 Internal Server Error:** Hubo un error general en el servidor.
- **502 Bad Gateway:** El servidor, al actuar como una puerta de enlace o proxy, recibió una respuesta no válida de un servidor upstream.
- **503 Service Unavailable:** El servidor no está disponible temporalmente, generalmente debido a sobrecarga o mantenimiento.
- **504 Gateway Timeout:** El servidor, actuando como puerta de enlace o proxy, no recibió una respuesta a tiempo.



How does HTTPS Work?

HTTPS

